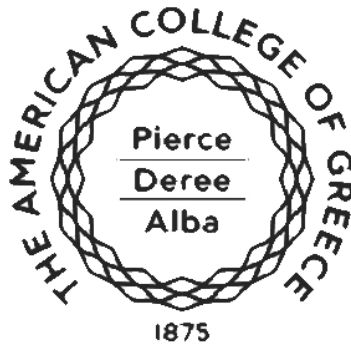




ITC 4214 A1: Internet Programming

Instructor: Dr. Mageiros Leonardos

Academic Term: Summer Term 2026

Dimitrios Kalligaridis (320738)

Dashboard for Defense Systems: A Responsive Task Manager

June 21st, 2026

1. Introduction

The task management dashboard I designed is called Argus Defense Systems and it is built for a defense contractor. It allows for a team to add program tasks, set a due date and priority for them as well as track them through to completion, while seeing a quick breakdown of the workload. The product includes 5 pages, Home, Tasks, About, Contact and a Capabilities page I selected to create, and it is built using HTML, CSS, JavaScript, jQuery and Bootstrap 5 loaded from a CDN. As per the instructions the project has no backend and thus all data is stored in the browser with localStorage, while the deployment is handled by GitHub Pages.

Live website: <https://retryoos.github.io/task-dashboard/>

GitHub Repo: <https://github.com/retryoos/task-dashboard>

2. How the task allocation system works

Every task is defined as a JavaScript object, and the board works on one array of the following objects.



Figure 1. The structure of a single task object.

A user allocates work by adding a task, setting a priority for it from Low, Medium and High, which are color coded and finally giving it a due date. A task starts as Pending as soon as it's created and is flipped to Completed when the button on its row is clicked. To shift focus onto what matters, the list can be filtered by status or priority and sorted by due date or alphabetically. A summary and two bar charts show how the work is split, and after every change the array is written to localStorage, so nothing is lost when the browser is refreshed.

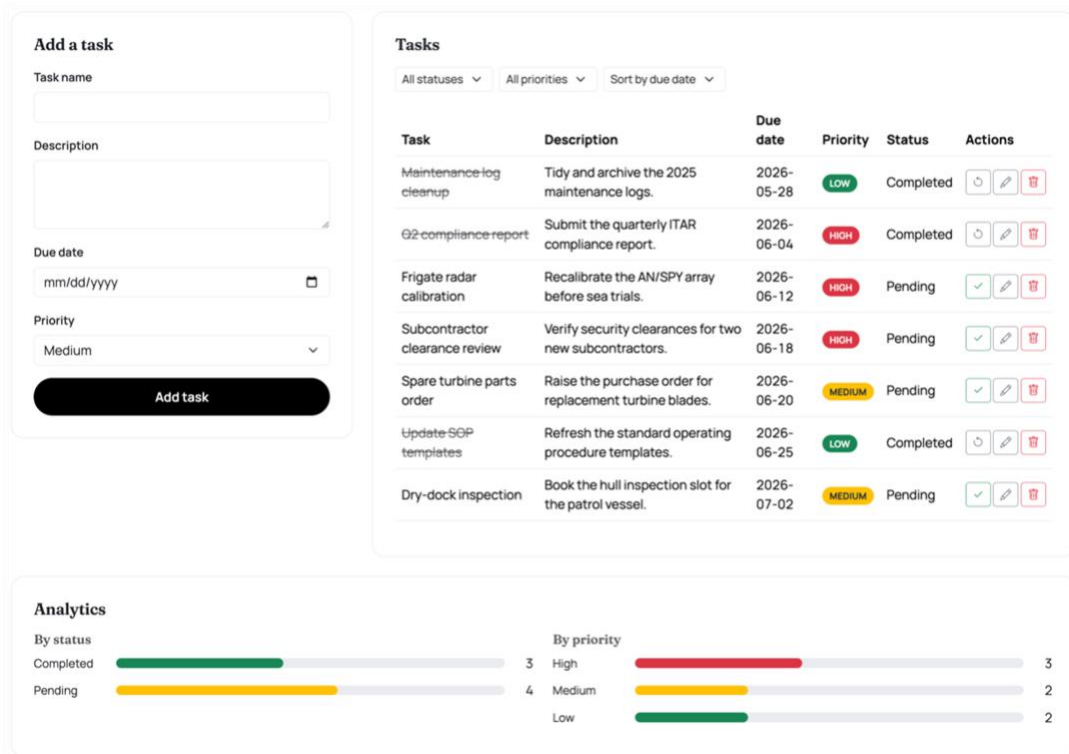


Figure 2. The Tasks page, with the add-task form, the task table, and the analytics charts.

3. Coding decisions

3.1. Tasks page

The table was built using `forEach` and template strings, and when a user adds a task its input is validated first with `required`, a minimum length and a maximum length cap. After the validation occurs then the object gets pushed into the array, while when the user edits it reuses the same form and updates the matching object instead of adding a new one.

The row buttons use event delegation, by using one handler on the table body rather than one per button, so they keep working on rows that are created after the page loads.



Figure 3. Deleting a task with event delegation, a single click handler on the table body.

Filtering does not change the real array, but by using a function called `getVisibleTasks` it returns a filtered, sorted copy, so no data is lost when the user switches filters. The analytics bars are plain divs whose width is set to a percentage (count / total), so no chart library was needed.



Figure 4. Filtering and sorting a copy of the task array, so the original list is never changed.

3.2. Latest activity section

Every add, edit, delete or complete action, records a short line in a second localStorage list which is capped at 20 entries. The Home page reads that list and shows the most recent ones, and this keeps the two pages in sync without a server.



Figure 5. Recording each action in a capped activity log stored in localStorage.

3.3. Personal page (capabilities)

This page shows 3 bootstrap components I did not use elsewhere, first it's the pill tabs for the divisions, second an accordion for the FAQ, and third a list group for resources. All run on Bootstrap's data-bs-* attributes, so the page needs to javascript of its own to run.

4. Key technical decisions

A few parts that went past the material we have covered so far in our course's lectures, and I would like to explain briefly are as follows. First saving state with localStorage and JSON were used because there is no server and I use JSON to store the task array and the activity list as JSON strings by using JSON.stringify to save, and JSON.parse to read back. This is what allows the dashboard to persist it's data after a refresh of the browser.

Furthermore, I escape any text the user types in so that it can never run as code. Since I build the task rows by joining strings together and placing them into the page, a task named something like would otherwise be treated as real HTML and would execute. To prevent this, every piece of user text first passes through a helper function I wrote called escapeHtml, which converts the characters <, >, &, " and ' into their HTML entity equivalents, so the text is always shown as plain text and is never

executed. On top of that, I limit how much can be typed into each field using the `maxlength` attribute.



```
function escapeHtml(value) {
  return String(value)
    .replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;").replace(/'/g, "&#39;");
}
```

Figure 6. The `escapeHtml` helper that turns special characters into HTML entities, so user input is shown as text and never run as code.

Lastly, there are a few smaller extras I included that also go a little past the basics we have learned so far. On the Home page I pull in live weather data for the HQ city (Athens) from the free Open-Meteo API using jQuery's `$.ajax`, and I show it in the hero page. I gave it both a success and an error handler, so the page still works even if the request does not return the necessary data. As to the dark mode I included I achieved it using Bootstrap's built-in `data-bs-theme` attribute, which the toggle button sets to "dark" on the `<html>` element and saves in `localStorage`, so Bootstrap changes the theme's appearance of the whole site on its own and remembers the choice on the next visit or refresh. To keep the layout flexible, the headings scale smoothly between phone and desktop using `clamp()`, while the stylesheet reuses Bootstrap's own theme variables such as `--bs-emphasis-color`, which keeps the CSS small and lets it theme correctly in both light and dark mode.

5. Performance and accessibility audit (Lighthouse reports)

To measure performance and audit the site I used Lighthouse on the Chrome dev tools on all five pages of my website for both desktop and mobile. The full reports are in the directory `LightHouse` folder and the scores are summarized on the tables below for readability.

Page Name	Performance	Accessibility	Best Practices	SEO
Home	83	95	100	100
Tasks	94	96	100	100
About	94	95	100	100
Contact	95	96	77	100
Capabilities	94	96	100	100

Table 1. Mobile LightHouse score summary.

Page Name	Performance	Accessibility	Best Practices	SEO
Home	98	100	100	100
Tasks	99	100	100	100
About	98	100	100	100
Contact	99	100	77	100
Capabilities	99	100	100	100

Table 2. Desktop LightHouse score summary.

Looking at the results, SEO came out at 100 on every page, which is as expected given each page has its own title, a meta description and a language attribute. Accessibility is 100 across all on desktop and drops only a little to 95 to 96 on mobile, where Lighthouse points out small things like tap-target spacing and slightly lower contrast at the smaller size. Performance is very high on desktop at 98 to 99 and stays good on mobile at around 94 to 95, with the one exception being the Home page on mobile at 83. The reason for that is the Home page loads the most from outside sources, the Bootstrap CSS and JavaScript, jQuery and 2 Google fonts, and on top of that it makes the live weather request, all of which costs more on a throttled mobile connection. Best Practices is 100 everywhere except the Contact page, which sits at 77 on both desktop and mobile, and this is down to the embedded Google Maps, since the maps are third-party content that pull in cookies and files I have no control over, since that is simply the trade-off for meeting the requirement to embed a Google Map.

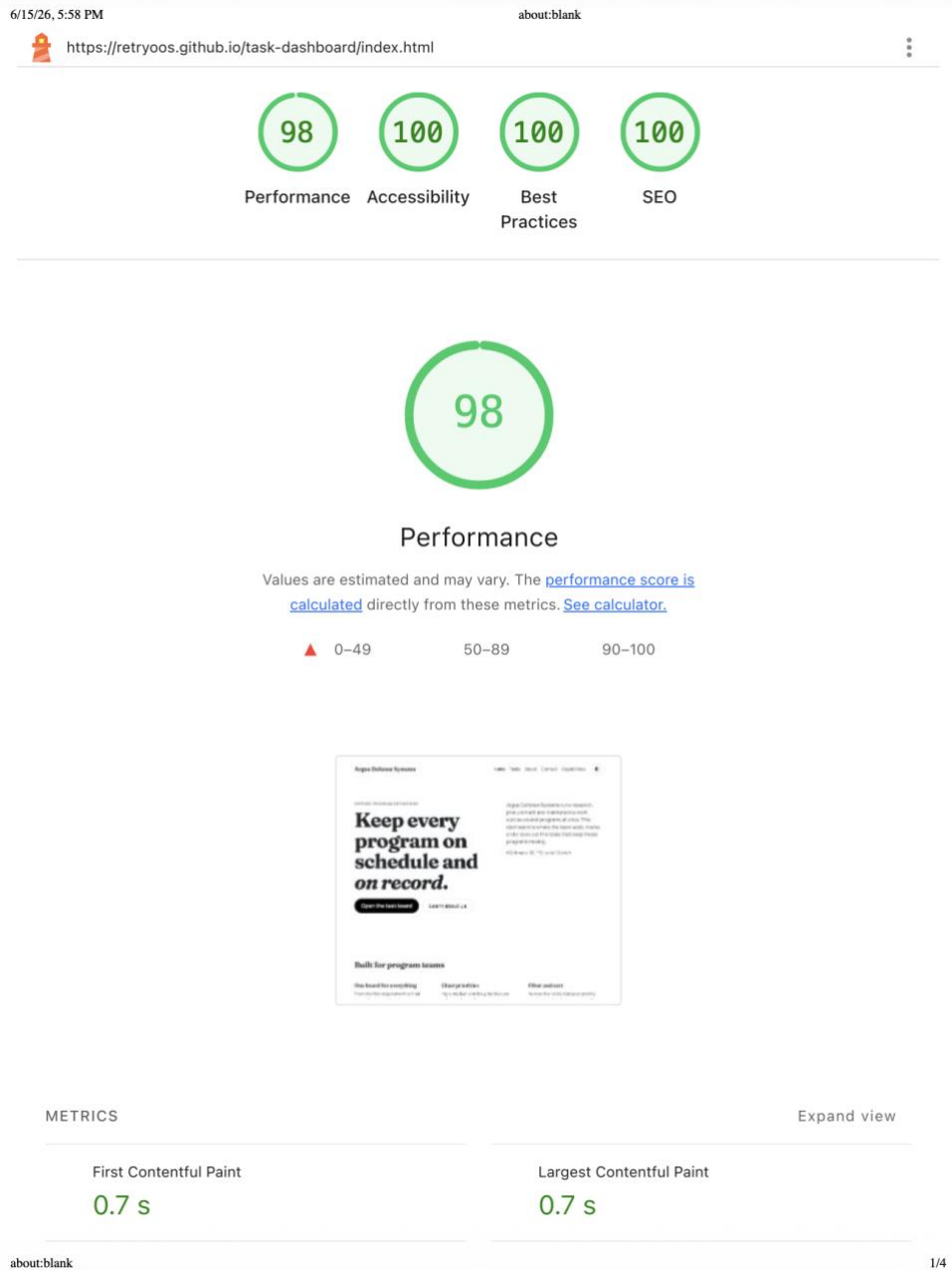


Figure 7. LightHouse Desktop Home page insights

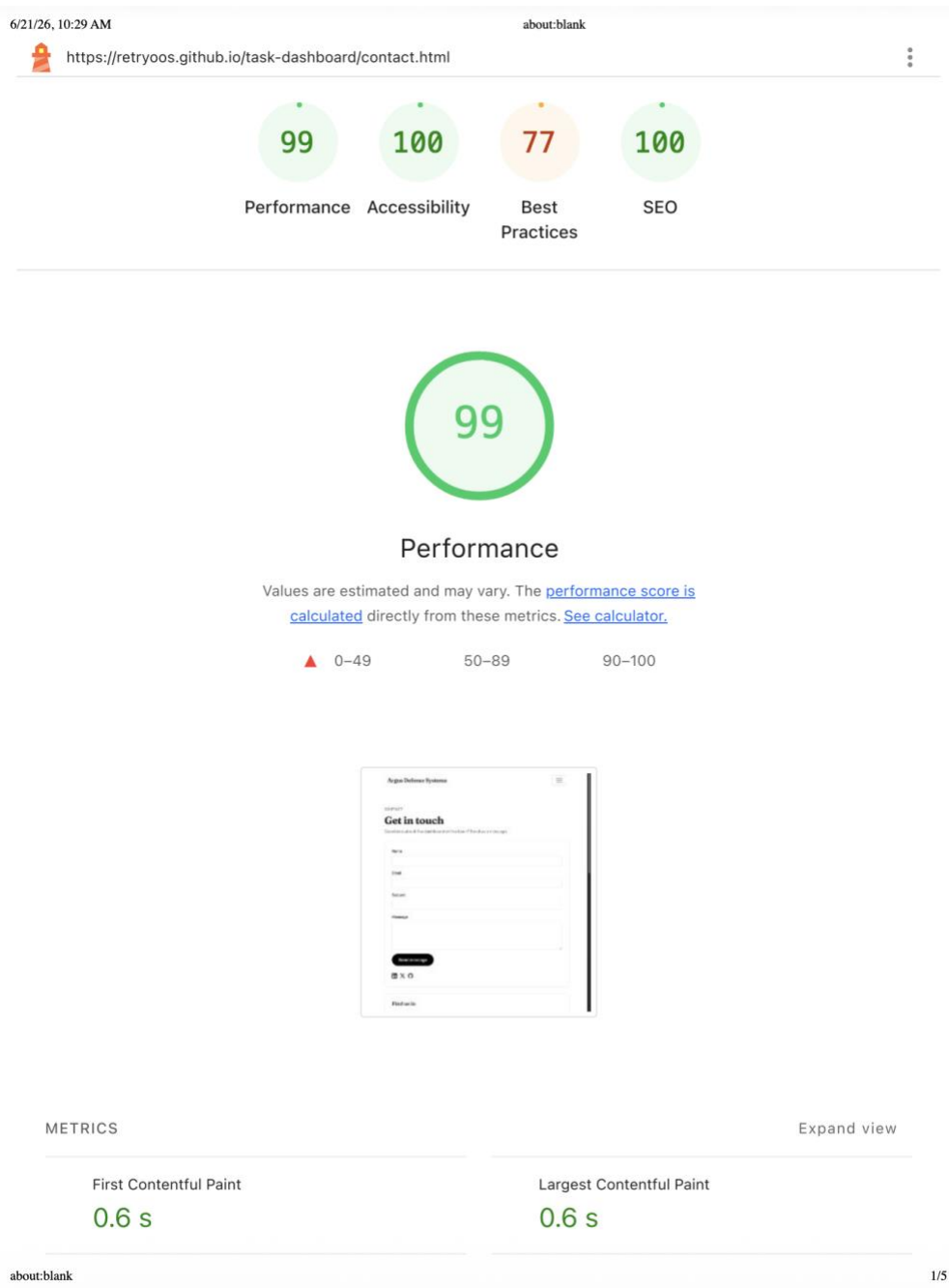


Figure 8. LightHouse Desktop Contact page insights

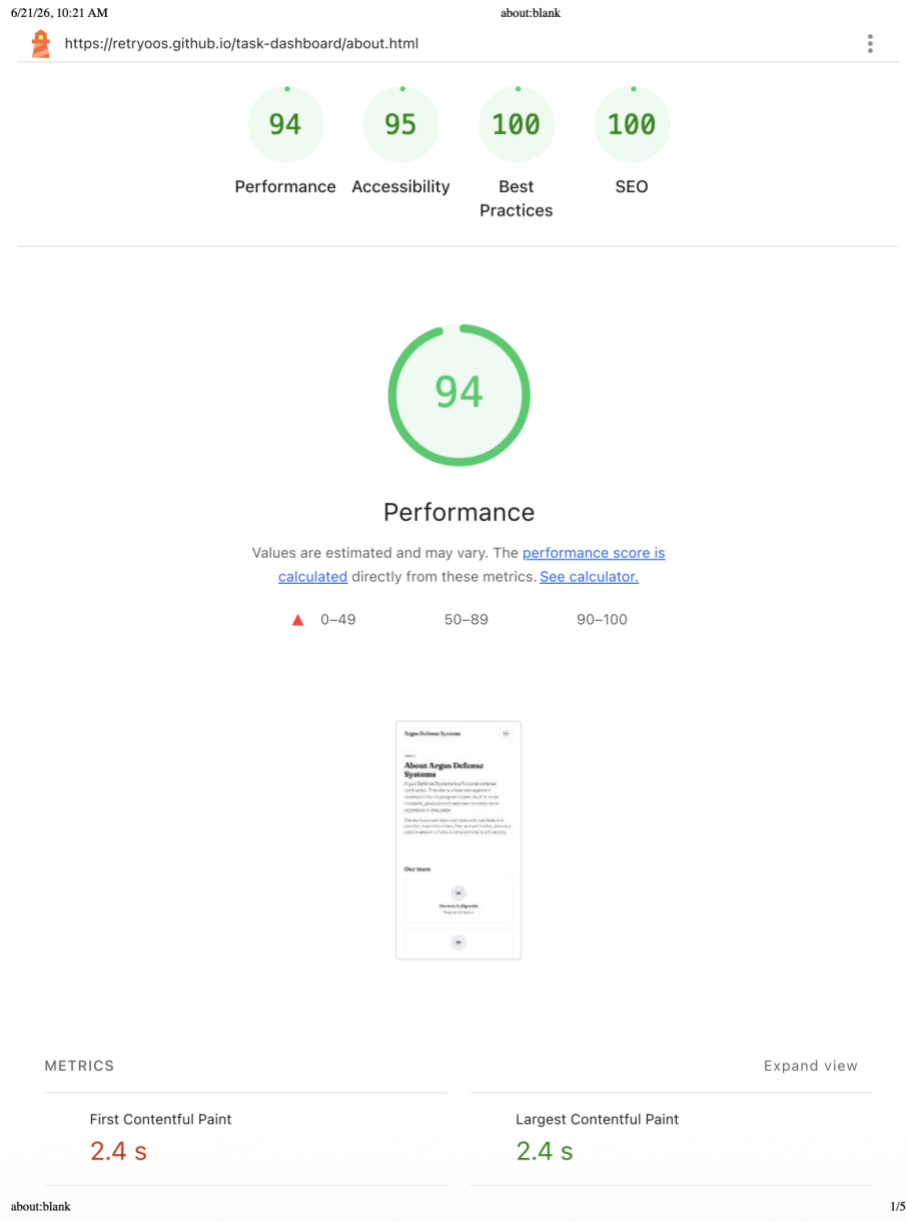


Figure 9. Lighthouse Mobile About page insights

6. Reflections and Challenges

To conclude, I think the scores are pretty good for a static site with no build steps, and if I had more time to push the mobile performance higher I would host the CSS myself and minify it, as well as preload the fonts. Lastly, to push the Lighthouse contact score higher I would replace the live map with a static map image, although that would mean losing the ability to pan and zoom through our website.

There were a few things that gave me trouble throughout the coding period of the project. The filtering took a few attempts, because at first I was changing the main task array while filtering it, which ended up dropping tasks, and I fixed it by making the filter return a copy and leaving the original list intact. The row buttons were another one since they only worked once until I switched to event delegation and attached a single handler to the table instead of one to each button. Finally, the Google Maps embed was the one thing that pulled the Lighthouse score down, which turned out to be a useful insight in the real cost of adding third-party content to a page.